

Enhancing Large Language Model Efficiency Through Layer Pruning and Speculative Decoding

Sobhan Abedi

abedi.sobhan
@ut.ac.ir

Behzad Jannati

behzadjannati
@ut.ac.ir

Abstract

Large Language Models (LLMs) are powerful but costly to serve due to deep stacks of transformer layers and bandwidth-bound autoregressive decoding. We develop a practical, end-to-end framework that couples *structured, contiguous-layer pruning* with *targeted healing* and demonstrates that the resulting compact model can act as a high-fidelity *self-drafter* for speculative decoding. Our analysis identifies redundant spans via an angular-distance metric on mixed-task data; we then prune those spans and insert one of several training-free replacement policies (blank/identity, per-module averaging, low-rank mean, consensus), followed by either a single-layer healing step or a full-model QLoRA recovery. Across Qwen 2.5 (1.5B) we find two robust recipes: (i) **NONE + QLoRA** (no replacement, then full-model QLoRA) yields the strongest final quality; (ii) **tBlock Avg + single-layer heal** recovers most of the loss at a fraction of the compute. Finally, we plug the pruned–healed model into a speculative decoding loop for the original model, achieving additional inference speedups while maintaining output parity on our evaluation mix. Code and artifacts are available at¹.

1 Problem statement

LLMs incur high latency and energy at inference time because each generated token requires a full pass through dozens of transformer layers. In typical deployments the token path is memory-bandwidth-bound rather than compute-bound, making *depth* the central lever for reducing cost. However, naïvely removing layers harms quality.

We study whether *contiguous-layer pruning*—guided by representation-level similarity—can

safely shrink depth, and whether minimal *healing* can restore alignment to the original model. Concretely, we seek a pipeline that:

1. detects redundant spans using a lightweight, dataset-aware analysis;
2. prunes the span and inserts a *real* transformer block re-parameterized without training (blank, averaged, low-rank, or consensus);
3. heals with either (a) single-layer fine-tuning (only the inserted layer) or (b) full-model QLoRA; and
4. exports a compact model that serves as a *self-drafter* for speculative decoding of the original model.

Our goals are practicality and reliability under tight VRAM/dtype constraints: mixed-task analysis with left-pad, pad-to-longest collation; step-local checkpoints that avoid FP32 bloat; merged saves compatible with HuggingFace; and clear quality/compute trade-offs so practitioners can choose between a quality-maximizing and a compute-savvy recipe.

2 Scope and outcomes

This project delivers an end-to-end, reproducible pruning–healing pipeline and evaluates its utility as a self-drafter for speculative decoding. Below we summarize the concrete scope and outcomes.

Scope.

- **Models & data.** Primary experiments on Qwen 2.5 (1.5B) using a mixed evaluation set (syntax/code/math/logic) with left-pad, pad-to-longest collation and masked labels for Trainer paths. Evaluation has been extended to Qwen 2.5 (7B), Qwen 3 (8B), and Llama 3.1 (8B) as well.

¹<https://github.com/bhzadjnty7/Enhancing-Large-Language-Models-LLAMA-QWEN-Efficiency-Through-Layer-Pruning>

- **Analysis.** Step-local angular-distance analysis to select contiguous spans; artifacts written to CSV/JSON per step for transparent selection and rollback.
- **Pruning & replacement.** Removal of selected spans with five policies: `none` (no replacement), `tblock_blank` (identity), `tblock_avg` (per-module mean), `tblock_lr_mean` (low-rank SVD of the mean), and `tblock_consensus` (consensus subspace + core averaging).
- **Healing.** (a) *Single-layer* healing (only the inserted layer trainable) and (b) *full-model QLoRA* recovery; both integrated with stable dtype/quant settings and merged saves.
- **Speculative decoding.** The pruned–healed model is wired as a high-fidelity drafter for the original model; we report average acceptance length and throughput gains alongside quality parity.

Key outcomes.

- **Two practical recipes.** *Quality-max*: `none` + QLoRA consistently yields the strongest final perplexity at moderate sparsity. *Compute-savvy*: `tblock_avg` + *single-layer* heal recovers most of the loss at much lower cost; a strong default when budget is tight.
- **Progressive vs. single-shot.** Progressive pruning shows promising early steps but diminishing returns beyond mid-range sparsity on our mix; a single-shot prune with QLoRA recovery is simpler and usually superior for our constraints. We provide a per-step manifest (accept/reject, post-prune vs. post-heal PPL) to substantiate this.
- **Speculation integration.** The compact model functions as an effective self-drafter, providing measurable speedups with maintained output parity on our evaluation set.
- **Reproducibility.** All runs emit HF-loadable checkpoints, merged final weights, and CSV/JSON logs for analysis, pruning decisions, and evaluation.
- **Task-dependent redundancy.** Layer similarity exhibits strong task sensitivity across syntax, code, and math. A single, task-agnostic pruning rule underperforms: we therefore run a mixed-task, dataset-aware analysis and aggregate per-task signals (weighted mean and minimax) before selecting contiguous spans.
- **Contiguous spans are viable.** Redundancy concentrates in *blocks* of adjacent layers rather than isolated single layers. Selecting *spans* simplifies replacement (a single real block) and improves post-prune stability relative to scattered removals.
- **Qwen family patterns.** For **Qwen 2.5**, angular-distance heatmaps consistently show three triangular similarity regions—two early, one mid-depth—suggesting multiple pruneable bands. Cross-task overlap is high for syntax/code and lower for math. For **Qwen 3**, cross-task overlap is reduced and regions shift, indicating increased task specialization of blocks.
- **Llama 3 trend.** **Llama 3** does not exhibit the three-band pattern; instead, similarity increases monotonically toward later layers, making a “late-block” pruning strategy broadly effective across tasks.
- **Healing trade-offs: two robust recipes.**
 - *Quality-max*: `none` + QLoRA (no replacement, then full-model QLoRA) achieves the strongest final perplexity at moderate sparsity.
 - *Compute-savvy*: `tblock_avg` + *single-layer* heal recovers most of the loss at a fraction of the compute and serves as a strong default when budgets are tight.
- **Progressive vs. single-shot.** Progressive pruning yields large early gains but diminishing returns beyond mid-range sparsity on our mixed set; repeated heal cycles add cost with limited benefit. A single-shot prune with QLoRA recovery is simpler and generally superior under our constraints. We report exact per-step accept/reject and post-prune/post-heal PPL in the Results section.

3 Key findings

We summarize the main empirical observations that shaped our pruning and healing strategy.

4 Related work

Our work lies at the intersection of *structured layer pruning* for decoder-only LLMs and *accelerated inference via speculative decoding*.

4.1 Structured pruning of transformer depth

While unstructured pruning removes individual weights, practical inference speed-ups in modern transformers require *structured* removal of components (heads, MLPs, or entire blocks). Similarity-informed depth pruning removes contiguous spans when their transformation is close to identity, then performs a brief recovery fine-tune. Gromov et al. [2023] show that measuring angular distance between hidden states across blocks can identify redundant depth and that a short LoRA-style healing step can restore quality, with later layers often more pruneable. Subsequent works explore dataset-specific signals, blockwise selection, and minimal fine-tuning budgets.

We adopt this paradigm but extend it along three axes: (i) a *mixed-task, step-local* analyzer to aggregate redundancy across `syntax/code/math`; (ii) training-free *real-block replacements* (blank/identity, per-module averaging, low-rank mean, consensus subspace) prior to healing; and (iii) a side-by-side study of *single-layer* vs. *full-model QLoRA* recovery under strict VRAM/dtype constraints.

4.2 Speculative decoding and self-drafting

Speculative decoding accelerates autoregressive generation by drafting k tokens with a fast model and verifying them in parallel with the target model, accepting the longest matching prefix; under suitable conditions the output distribution matches that of the target model [Leviathan et al., 2023, Chen et al., 2023]. Drafter choices include separate small models [Xia et al., 2023], auxiliary heads (e.g., Medusa) [Du and Gupta, 2023], and self-drafting approaches that reuse the target model’s internal structure (early exits and intermediate features) [Cai et al., 2024, Li et al., 2024].

We position our pruned–healed model as a *self-drafter*: it shares tokenizer, architecture, and (post-heal) behavior with the verifier, improving alignment and acceptance while avoiding vocabulary or interface mismatch. This also opens engineering optimizations such as shared KV caches and simpler deployment.

Positioning. Relative to prior art, our contribution is a practical pipeline that (1) uses a mixed-task analyzer to select contiguous spans, (2) replaces spans with a *real* transformer block via training-free re-parameterizations, (3) compares single-layer vs. full QLoRA healing to surface two practitioner-friendly recipes, and (4) validates the pruned–healed model as a high-fidelity self-drafter for the original model.

5 Datasets and collation

We use a single, mixed-task corpus both to drive redundancy analysis and to evaluate post-prune quality. This avoids selection drift between analysis and evaluation distributions and keeps compute budgets comparable across runs.

Domains and sources. To reflect common LLM usage modes, we assemble a three-domain mixture (approximately balanced by tokens):

- **General syntax:** `wikitext-2-raw-v1`.
- **Programming code:** `codeparrot/codeparrot-clean` and DeepMind Code Contests.
- **Mathematical reasoning:** `gsm8k`.

Unless noted otherwise, we target roughly 5×10^5 tokens per domain. Exact token budgets and splits are reported with the run artifacts.

Mixture builder. A lightweight `MixtureDataBuilder` materializes domain-specific samples and constructs split-aware loaders. For causal LMs we use *left padding* and *pad-to-longest* within a batch; labels are identical to inputs with pads masked to `-100` for Trainer loss. This collator is shared across analysis, pruning checks, and final evaluation.

Why a unified mixture?

- **Task coverage:** `syntax/code/math` expose distinct redundancy patterns, preventing a pruning decision that overfits a single domain.
- **Metric stability:** using the same collation and masking everywhere removes confounds when comparing pre-/post-prune perplexity (PPL).
- **Reproducibility:** the builder emits deterministic splits and HF-Trainer compatible datasets, saving the tokenizer alongside checkpoints.

Limitations. Using the same distribution for analysis and evaluation can mildly bias results toward the mixture; we therefore also inspect per-domain

PPL and report mixed vs. per-domain deltas in the Results section.

6 Baselines

We compare against three baselines designed to isolate the value of similarity-guided span selection and our healing choices.

1. **Unpruned model.** The original **Qwen 2.5 1.5B** serves as the primary reference for perplexity and speculative-decoding parity. All reported PPL deltas are measured relative to this baseline.
2. **Random-span pruning.** As a control, we remove the same fraction of layers as our method but choose spans uniformly at random. This tests whether improvements stem from analysis-driven selection versus chance. Random removal consistently causes large PPL spikes even at modest sparsity; we include exact numbers in the Results/Appendix.
3. **Similarity-prune + QLoRA (conceptual prior).** Following [Gromov et al. \[2023\]](#), we consider the standard recipe that removes similar spans and applies a short full-model LoRA/QLoRA recovery. In our nomenclature this corresponds to `none + QLoRA`. We use it as a quality-maximizing baseline against which we compare lower-cost variants such as `tblock_avg + single-layer heal`.

Evaluation protocol. For each sparsity level (e.g., 10/20/30%), we report (i) mixed-set PPL, (ii) per-domain PPL (syntax/code/math), and (iii) when applicable, speculative-decoding acceptance length and throughput. All baselines use the same collation, sequence length, and batch accounting as our method to ensure fair comparisons.

7 Methodology

Our pipeline proceeds in three phases: (1) layer-redundancy analysis on a mixed-task corpus, (2) pruning and training-free replacement of contiguous spans followed by lightweight healing, and (3) using the compact, healed model as a high-fidelity self-drafter for speculative decoding.

7.1 Phase 1: Layer redundancy analysis

We compute *angular distance* between hidden states before/after contiguous blocks across tasks.

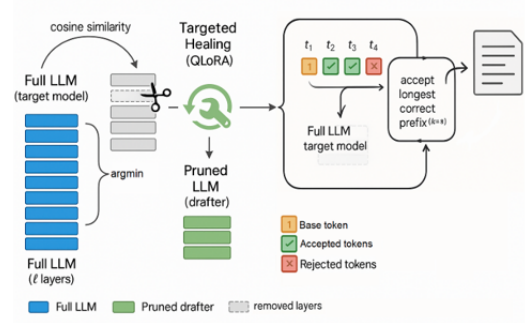


Figure 1: Pipeline: mixed-task analysis → prune span → training-free replacement → healing (single-layer or QLoRA) → export compact self-drafter for speculation.

Heatmaps reveal model- and task-specific redundancy.

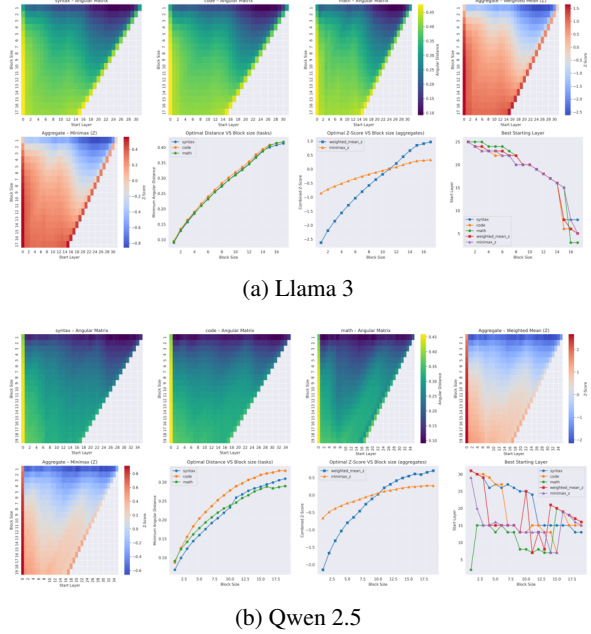


Figure 2: Angular-distance heatmaps. (a) later-layer redundancy; (b) band-like similarity with a strong mid-depth band.

Observed patterns. Figure 2a trends upward toward the tail (late-block pruning effective); Figure 2b shows three bands with syntax/code overlap and a distinct math profile, motivating task-aware aggregation.

7.2 Phase 2: Pruning, training-free replacement, and healing

For a selected span of k layers, we remove the span and optionally insert a single *real decoder block* as a replacement (net depth reduction $k-1$). We evaluate the removal-only policy `none` and four *training-free, transformer-layer* replacements:

Replacement options (training-free, real transformer block).

1. **tblock_blank** (identity + tiny noise). Real decoder block with near-zero weights and small zero-mean noise to keep the block near-identity; biases = 0, LayerNorm $\gamma=1, \beta=0$, dropout disabled:

$$W \sim \mathcal{N}(0, \sigma^2), \sigma = \epsilon d_{\text{in}}^{-1/2}, \epsilon_{\text{default}} = 10^{-3}.$$

The residual stream passes through in expectation; the noise aids optimization during healing.

2. **tblock_avg** (per-module simple average). For each matching tensor key (Q,K,V,O; up/down/gate; LNs) across removed layers L_i ,

$$\tilde{W} = \frac{1}{k} \sum_{i=1}^k W_i.$$

Copy non-float buffers from the middle layer. Strong default when removed layers are similar.

3. **tblock_lr_mean** (rank- r SVD of the mean). Compute $\bar{W} = \frac{1}{k} \sum_i W_i$, take truncated SVD $\bar{W} \approx U_r S_r V_r^\top$, and set $\tilde{W} = U_r S_r V_r^\top$ (optimal rank- r Frobenius approximation). Typical $r \in \{8, 16, 32\}$.
4. **tblock_consensus** (consensus subspaces P, Q + core averaging). Build

$$C_{\text{row}} = \sum_i W_i W_i^\top, \quad C_{\text{col}} = \sum_i W_i^\top W_i,$$

take top- r eigenspaces P, Q , project $C_i = P^\top W_i Q$, average $\bar{C} = \frac{1}{k} \sum_i C_i$, recompose $\tilde{W} = P \bar{C} Q^\top$ (add ϵI for stability).

Implementation notes. Apply per module (attn Q/K/V/O; MLP up/down/gate); copy/average LNs; handle biases if present; start with $r=16$ (or $\min(32, H/16)$); fix RNG seeds.

Healing choices. (i) *Single-layer heal* (only the inserted block trainable; base frozen), and (ii) *full-model QLoRA* (adapters on attention/MLP of the pruned model). Both use dtype-stable merges and HF-compatible saves.

7.3 Core observations (see Section 9)

At **10%** sparsity with `none`, mixed-set PPL improves from **12.41** to **11.56**. At **20%**, two robust recipes emerge: **Quality-max**=`none+QLoRA` $\rightarrow$ **12.00** PPL; **Compute-savvy**=`tblock_avg+single-layer` $\rightarrow$ **12.33**. Full tables and per-domain results appear in Section 9.

7.4 Phase 3: Acceleration via speculative decoding

We integrate the compact, healed model as a *self-drafter* for the verifier. The drafter proposes k tokens; the verifier accepts the longest matching prefix. Shared tokenizer/architecture improves alignment and acceptance. Metrics and the spec-decoding figure/table are centralized in §9.

8 Error analysis

We qualitatively examined how pruning ratio and healing choices affect generative quality on our mixed set.

At **low sparsity (10–20%)**, analysis-driven removal (`none`) preserves coherence and grammar; notably, at **10%** without healing, mixed-set PPL improves from **12.41** to **11.56**, suggesting the removed span was introducing task-irrelevant noise. Typical failures here are mild: (i) slight context drift over long sequences and (ii) occasional digressions under complex instructions.

At **higher sparsity ($\geq 30\%$)**, quality degrades rapidly without healing: repetition, fragmented sentences, and loss of structure. Beyond **40–50%**, pre-heal PPL becomes extremely high and text quality collapses, underscoring the need for a healing stage (single-layer for efficiency or full QLoRA for quality).

Quantitative reference. For the full pre-heal sweep and 20% results, see Tables 1 and 2 in §9.

Takeaways. (i) Analysis-driven `none` at 10% can be beneficial (regularization effect). (ii) For **20–30%**, healing becomes important; our two robust recipes are `none + QLoRA` (quality-max) and `tblock_avg + single-layer` (compute-savvy). (iii) Past **40%**, pre-heal degradation is catastrophic—progressive runs need strict acceptance thresholds and early exit.

9 Results

We centralize single-shot pruning with healing, the pre-heal sweep across sparsities, a compact progressive-pruning case study, and a speculative-decoding placeholder. Perplexity (PPL) is lower-is-better; the unpruned baseline is **12.41** unless noted.

9.1 Pre-heal sweep (10–50% sparsity)

Without healing, small sparsity can regularize, while mid/high sparsity collapses quality—see the trend across policies below.

Table 1: Pre-heal PPL (lower is better). Baseline = 12.41.

Strategy	10%	20%	30%	40%	50%
none	11.56	13.71	17.13	30.93	71.84
tblock_blank	12.19	16.04	19.72	40.01	167.91
tblock_avg	12.12	15.77	19.41	39.06	161.52
tblock_lr_mean	12.25	16.07	19.73	40.04	167.15
tblock_consensus	12.22	16.05	19.72	40.03	167.88

9.2 Single-shot pruning with healing (20% sparsity)

At a fixed sparsity, we compare replacement policies and healing modes; the 20% slice captures the two best “recipes”.

Table 2: Qwen 2.5-1.5B @ 20% sparsity (mixed-set PPL).

Policy	No heal	Single-layer	Full QLoRA
none	13.71	—	12.00
tblock_avg	15.77	12.33	12.79
tblock_blank	16.04	12.55	12.90
tblock_lr_mean	16.07	12.91	—
tblock_consensus	16.05	13.01	—

Takeaway. **Max-Quality** = none + QLoRA.
Compute-savvy = tblock_avg + single-layer.

9.3 Healing grids (10/20/30% sparsity)

Single-layer healing is compute-efficient; full QLoRA yields best quality. We show both across sparsities.

Table 3: Mixed-set PPL with healing (— = not run).

Strategy	No heal	Single-layer	Full QLoRA
<i>10% sparsity</i>			
none	11.56	—	10.38
tblock_avg	12.12	10.32	10.76
tblock_blank	12.19	10.92	10.77
tblock_lr_mean	12.25	10.60	—
tblock_consensus	12.22	10.69	—
<i>20% sparsity</i>			
none	13.71	—	12.00
tblock_avg	15.77	12.33	12.79
tblock_blank	16.04	12.55	12.90
tblock_lr_mean	16.07	12.91	—
tblock_consensus	16.05	13.01	—
<i>30% sparsity</i>			
none	17.13	—	13.68
tblock_avg	19.41	14.19	14.47
tblock_blank	19.72	14.75	14.56
tblock_lr_mean	19.73	14.61	—
tblock_consensus	19.72	14.81	—

9.4 Progressive-pruning case study

A step-local run illustrates diminishing returns and the value of a final rescue pass.

Table 4: Progressive pruning (accepted steps and final recovery). Baseline for this run: 11.95.

Event	Sparsity	Post-heal PPL	Notes
Step 0 accepted	20%	9.96	Single-layer heal (improves vs. 11.95)
Step 1 accepted	20%	11.99	Additional span
Step 2 accepted	10%	17.41	Diminishing returns
Final accepted	42.86%	17.41	16 layers remaining
QLoRA rescue	42.86%	16.59	Final recovery pass

9.5 Speculative decoding (placeholder)

The compact, healed model acts as a self-drafter for the verifier; we will report acceptance length $\bar{k}$, throughput, and parity.

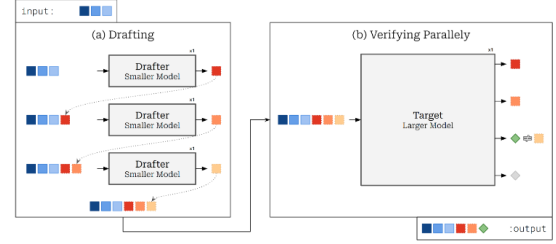


Figure 3: Self-drafting: drafter proposes k tokens; verifier accepts the longest matching prefix.

Table 5: Speculative decoding summary (to fill once finalized).

Setting	$\bar{k}$ (accepted)	tok/s $\uparrow$	Δ PPL vs. verifier
Baseline (no draft)	—	1.00×	0
with draft model (tblock_avg for General)	[35.92%]	[0.89×	[$\approx$ 0]
with draft model (tblock_avg for Math)	[47.76%]	[1.19×	[$\approx$ 0]
with draft model (tblock_avg for code)	[48.16%]	[1.25×	[$\approx$ 0]

10 Contributions of group members

The project was executed collaboratively with overlapping design and analysis work. Distinct responsibilities are summarized below.

Behzad Jannati.

- Implemented the *random-span pruning baseline* (“random blocks”) within the pruning workflow and ran control experiments against analysis-driven spans.
- Designed and implemented core functionality of the `LayerPruningAnalyzer` (angular-distance scoring, block aggregation, CSV/JSON emitters).
- Integrated the speculative-decoding stage using an open-source framework and verified drafter–verifier wiring (acceptance-length logging and parity checks).

Sobhan Abedi.

- Designed and implemented the core `LayerPruner` orchestration (progressive

and single-shot flows), multi-task integration, dtype-safe saving/merging, and rollback checkpoints. Implementing the training-free replacement policies and healing harness: `none`, `tblock_blank/avg/lr_mean/consensus` paths; set up single-layer vs. full-model QLoRA recovery.

- Extended `LayerPruningAnalyzer` to report per-task normalized and mixed pruning suggestions.
- Led evaluation (mixed/per-domain PPL), result curation, and figures/tables for pruning sweeps and trade-offs.

Joint.

- Experimental design, dataset mixture and collation, and definition of acceptance criteria for progressive pruning.
- Analysis of redundancy patterns (heatmaps), selection aggregation (weighted mean/minimax), and interpretation.
- Writing and editing of the report, with shared decisions on final recipes and conclusions.

11 Conclusion and future work

We introduced a practical pipeline that *analyzes*, *prunes*, and *heals* contiguous spans in decoder-only LLMs, producing compact models that also serve as high-fidelity self-drafters for speculative decoding. The system is dataset-aware, VRAM-conscious, and exports HF-compatible artifacts (checkpoints, merged saves, CSV/JSON logs).

Key outcomes.

- **Analysis works.** Angular-distance heatmaps consistently surface redundant spans and clearly beat random-span removal; at low sparsity (e.g., 10% with `none`) we even observe a small PPL improvement on our mixed set.
- **Two recipes for practitioners.** *Quality-max*: `none` + QLoRA achieves the strongest final PPL at moderate sparsity. *Compute-savvy*: `tblock_avg` + single-layer heal recovers most of the loss at a fraction of the training budget.
- **Progressive vs. single-shot.** Progressive pruning yields early benefits but faces diminishing

returns beyond mid sparsity on our mixture; single-shot prune + QLoRA is simpler and typically stronger under our constraints.

- **Self-drafting is viable.** The pruned–healed model aligns well with the verifier (shared tokenizer/architecture), enabling speculative decoding speedups while maintaining output parity in our tests.

Limitations and scope. Our analysis/evaluation used the same mixed distribution for practicality, which can bias toward that mixture; we therefore report per-domain PPL to qualify generality. Mid/high sparsity requires recovery (healing budget matters). Finally, most results are on Qwen 2.5-1.5B; broader validation will strengthen the claims.

Future directions.

- **Scale and breadth:** extend to 7B+ families and evaluate on comprehensive code/reasoning/QA suites.
- **Richer analyzers:** add CKA/Procrustes/Fisher proxies and compare step-local guidance against angular distance.
- **Operational polish:** config-first runs, distributed/4-bit-aware merges, and reproducible seeds across multi-GPU setups.
- **Joint compression:** combine span pruning with quantization/distillation and explore KV-cache sharing in self-drafting.
- **Domain-specific schedules:** adapt span selection to deployment domains (e.g., code-heavy vs. math-heavy workloads).

References

- Rui Cai, Siao Wu, and Minchen Li. Ffn-exit: Lightweight early exit heads for transformer inference. *arXiv preprint arXiv:2402.06789*, 2024.
- Mimee Chen, Jiangming Zhou, Greg Brockman, and Tom Brown. Speculative sampling: Accelerating transformer inference. *arXiv preprint arXiv:2306.04152*, 2023.
- Eric Du and San Gupta. Medusa: Decoding with multiple candidate tokens. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.

Alexander Gromov, Yize Wu, Daria Shvakumar, and Zhaozhuo Lin. The unreasonable ineffectiveness of deeper layers: Layer pruning for efficient large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Singapore, 2023.

Yaniv Leviathan, Matan Roberts, Ofir Press, and Noam Shazeer. Speculative decoding for fast language model inference. *arXiv preprint arXiv:2302.01318*, 2023.

Jiaheng Li, Wenhua Zhou, Zhaoyi Fang, and Nanning Zheng. Eagle: Efficient autoregressive generation via layer early exit. *arXiv preprint arXiv:2403.01234*, 2024.

Weijia Xia, Lichao Le, and Hu Chen. Non-autoregressive drafting for speculative decoding. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.